Technical Report

Submission-to-Decision Traceability Platform

Sydney Erik Wamalwa · 7 November 2026

1 Scope Alignment and Feature Validation

The approved workflow is implemented end to end. Citizens send views on tracked bills and receive hash-chained receipts. Committee reports are read and split into entries, an AI proposes which entry answers each view, and a moderator confirms or rejects every proposal before any outcome is shown. Tracing deliberately stops at the **committee report**: that is where a legislature formally records what it heard from the public, so it is the evidence for whether citizens' voices were heard. Coverage is the National Assembly and the two pilot counties, Nairobi City and Trans Nzoia. **Status** below separates local validation from deployment. The hosted API runs this release's code and schema, and demo data has been removed from the hosted database.

Planned feature	Purpose	Status	Evidence / caveat
National Assembly tracking	Bills, current stage and dated history from Parliament's official Bills Tracker.	Implemented, tested	`scripts/sync_legislation.py`. Parsing "NA."/"Sen." references raised parsed tracker rows from 194 to 322; local sync: 592 bills, 0 skipped.
County assemblies	Nairobi and Trans Nzoia bills, their Acts on Kenya Law, and county committees.	Implemented, tested	`scripts/sync_county_bills.py`, `committee_assignment.py`: 104 + 4 records, 28 committees. Assemblies publish no bill stages, so none are invented.
Participation and submission	Accept views only while a committee takes them; one view per bill, editable; memoranda upload.	Implemented, tested	`participation.py` (call-for-memoranda window, else First Reading only); `SubmitViewPage.tsx`; `test_submissions.py`.
Receipt integrity	Append-only SHA-256 chain; edits as new versions; full and single-receipt verification.	Implemented, tested; hosted chain valid	`receipt_service.py`, database triggers. Hosted: demo receipts removed and the chain resealed under an audit-log entry; 10 of 10 receipts verify.
Participation notices	Parliament's calls for memoranda, with notice date, window, mode and bill-text access.	Implemented, tested	Site search and landing pages (`discover_notice_pages`); one notice per bill per page; filters, sort and 10-per-page table in `NoticesPage.tsx`. Local: 34 notices.
Reports, matching, review	Extract committee treatments, propose links, reserve the decision for a moderator.	Implemented; Gemini check 4/4	`ingestion_service.py`, `llm_matching_service.py`, `report_worker.py`, `ModeratorPages.tsx`. Four-case check on a real 95-entry report passed; accuracy on a labelled set not yet measured.
Stages and outcome trail	Plain-language journey from received to the committee's response.	Implemented, tested	`submission_stage.py` / `submissionStage.ts`; bill, receipt and role home pages.
Roles and access	Role home pages; Clerk's Office limited to one committee; abuse limits.	Implemented, tested	`ensure_committee_scope`, `rate_limit.py`, `AppContext.tsx` scoping; `test_security.py`.
Offline drafts (FR10)	Draft during poor connectivity and send later.	Implemented	`utils/offline.ts`; drafts the API rejects permanently are dropped instead of blocking the queue.

Outside this release: forwarding submissions to committee clerks (so the platform cannot yet prove delivery), tracking bills past the committee report, an interface in Kiswahili (notifications are bilingual), and assemblies beyond the two pilot counties.

2 Technical Functionality and Architecture

A React/TypeScript (Vite) client on Netlify calls a FastAPI REST API on Render backed by PostgreSQL on Neon. Requests pass an in-memory rate limiter, JWT authentication and role checks; Clerk's Office actions also pass a committee-scope check. Routes delegate to domain modules: `participation` (when a bill accepts views), `submission_stage` (a submission's journey, mirrored in the client), `receipt_service` (chain, versions, verification), `ingestion_service` (`pdftotext` with Tesseract OCR fallback), `linking_service` and `llm_matching_service` (Gemini proposals), `committee_assignment`, `submission_documents` and `notification_service` (English/Kiswahili). Two sync scripts load data from official sources and accept only same-domain links; one county site carries injected spam links. A single background worker processes linked reports, so page views never trigger OCR or AI calls.

Receipts are protected by database triggers that reject UPDATE and DELETE. An edit is therefore a new row (`<ID>-v2`) carrying the citizen's original receipt ID, and each hash also covers an uploaded document's SHA-256. Gemini receives its rules as a system instruction and citizen text as data; the API key travels in a header, never the URL, and busy responses (429/500/503) are retried with back-off. Scores are capped, and moderators are warned, when a submission reads like instructions to the AI or the wording does not support the match. Committees come from official evidence where available, otherwise from the bill's subject; administrators confirm the latter, and the sync never overrides them.

3 Testing Approaches and Results

1. Automated checks. `npm run typecheck` and `npm run build` pass. The main bundle is 1,345 kB (393 kB gzip), above Vite's 500 kB advisory. `npm audit` reports 5 advisories (3 moderate, 2 high) in react-router, vite, uuid and source-map-js. The React Router open-redirect case is mitigated in the sign-in redirect helper. The backend suite gives **92 passed, 11 warnings** (third-party deprecations) in about 135 s, run against PostgreSQL with the production append-only triggers installed. It covers:

- tamper detection (edited text, a deleted receipt, a swapped document) and append-only edits;
- participation windows, one submission per bill, and a full stage walk-through;
- committee scope, sign-in lockout and the global limit, and AI-score capping;
- live tracker formats and the county parsers.

An earlier report of 3 failures came from the evaluation fixtures not being mounted in the container. The fixtures are now mounted.

2. Browser end-to-end (local, Playwright, Chromium). The tests completed these journeys with no console errors:

- a citizen redirected to sign in and returned to the bill, then submitting, viewing the paginated table and dialog, editing (same receipt ID), and verifying;
- a Clerk's Office account signing up for Nairobi Health Services, being approved, and seeing only its 5 bills;
- the moderator queue tabs, and the administrator's committee-assignment screen;
- sign-in, sign-up and reset pages without the navigation menu, which appears after sign-in;
- the notices table: status counts, search ("finance" leaves 3 of 34), legislature and mode filters, and 4 pages of 10 that reset to page 1 when a filter changes.

No horizontal overflow was found at 390 px.

3. Gemini matcher check (6 November 2026). `evaluation/evaluate_gemini_matcher.py` sent four test submissions to `gemini-3.5-flash-lite` against the Finance Committee's 95-entry report on the Virtual Asset Service Providers Bill, 2025. All 4 passed in 4.9 s: a direct Clause 14 submission matched Clause 14 (0.95); a paraphrase with no clause matched the public-register entry under Clause 18(1) (0.90); an unrelated pothole complaint and a prompt-injection attempt got no proposal. This is a smoke test, not an accuracy measure. A first attempt returned HTTP 503, so busy responses are now retried.

4. Hosted checks (6 November 2026). Netlify returned 200 in 1.6 s, Render `/health` 200 in 21.9 s (a cold start), `/api/v1/bills` 200 in 6.2 s, and `/receipts/{id}/verify` 200 in 1.6 s, so the hosted API runs this release. A read-only database check found the schema at the latest migration (`20261106_notice_per_bill`). The chain failed at 5 of 29 receipts, all demo receipts from the seed script. **Cleanup (7 November):** after a `pg_dump` backup, one transaction removed 10 demo and test accounts, 5 demo bills, 19 receipts on them or by those accounts, 4 reports, 30 messages and 11 demo MP profiles. It returned 10 matches reviewed by demo moderators to the queue, kept all audit history (demo actors blanked), resealed the 10 remaining receipts and logged the old and new chain heads. It committed only after the chain verified. Hosted `/receipts/verify` now returns valid, 10 of 10; the demo administrator can no longer sign in, and the seed script refuses non-local databases. **Next steps:** re-run the syncs on the hosted database (it holds 4 notices against 34 locally), confirm committee assignments, and evaluate matching accuracy and usability on approved data.

4 Code Quality Practices

Concerns are separated: typed pages and shared components for presentation, `src/api/` for HTTP, routes for coordination, models for persistence, and services for domain logic. Rules that both sides need are defined once per side with the same vocabulary: the submission stages, and the participation reason the API sends to the client. Tests sit at the trust boundaries: chain integrity, access scope, rate limits, untrusted parser input, and AI output. They use production-like database constraints, so a bug that only appears with the triggers is caught (one was, and fixed in this release). Sync code is defensive: official columns are read in preference to free text, ambiguous rows are skipped rather than guessed, and every parser is covered by tests on real document formats.

Quality risks remain:

- Large modules: `routes.py` is about 1,900 lines and should be split by domain.
- Bundle and start-up size: the bundle is large, and the app loads about 2.3 MB of data at start-up.
- Scaling: rate-limit counters live in memory, so they need Redis before the API runs as several instances.
- Dependencies: the npm advisories are still open.
- Matching accuracy: four cases pass on a real report, but precision and recall need a labelled set.

Evidence basis: repository source and tests at 6 November 2026; local Docker Compose runs (backend tests, Playwright journeys, the Gemini check, data syncs against parliament.go.ke, county assembly sites and Kenya Law); hosted HTTP and database checks, and the hosted cleanup, on 6–7 November 2026. Counts and timings are point-in-time observations.